

爱团本地生活服务综合平台技术总结报告

1. 摘要

爱团本地生活服务综合平台面向本地生活服务场景，围绕用户消费、商家经营和平台治理三类角色，建设用户端、商家端、管理端、后端服务、数据库与部署运维链路。项目以需求、设计、实现、测试、部署和过程管理的完整闭环为目标，在原有系统基础上补齐业务场景清单、三层建模、测试追溯、容器化、CI/CD、Kubernetes 部署方案以及微服务拆分设计与工程骨架。

本报告重点总结软件工程实践中的需求建模、架构设计、质量保障、持续交付、微服务拆分、云原生实验设计与项目管理过程。具体业务功能仅作为工程实践载体进行概括，不对页面和接口实现细节作过多展开。

2. 项目概况

2.1 系统定位

爱团是一个本地生活服务综合平台，覆盖外卖订餐、到店团购、生活服务、休闲娱乐等服务类型，提供从服务发现、交易下单、履约使用、评价售后到平台治理的端到端流程。系统目标不是单一页面原型，而是具备需求文档、设计模型、可运行代码、数据库迁移、自动化测试、部署脚本和持续集成流水线的完整工程。

2.2 用户角色与系统边界

系统主要包含三类业务角色：

角色	主要诉求	系统响应
用户	查找服务、下单支付、使用权益、评价与售后	用户端 APP / Web 支持发现、搜索、交易、会员、客服、投诉等流程
商家	管理门店、维护商品服务、处理订单、服务用户	商家端 Web 支持门店资料、商品服务、履约、核销、评价客服等操作
平台	管理商户、治理内容、处理纠纷、查看运营数据	管理端 Web 支持用户商户治理、订单治理、评价投诉、配置审计等能力

2.3 技术栈概览

层次	技术选型	说明
用户端	Flutter / Dart	支持移动端 APP 与 Web 预览，承担用户消费主流程
商家端	Vue 3 + TypeScript + Vite	承担商家经营和履约操作
管理端	Vue 3 + TypeScript + Vite	承担平台治理、配置、审计和看板功能
后端	Spring Boot 3 + Java 17	提供 REST API、权限边界、业务编排和数据访问

层次	技术选型	说明
数据库	MySQL 8 / H2 MySQL Mode	MySQL 用于部署环境，H2 用于本地与 CI 测试兼容验证
数据迁移	Flyway	支持空库初始化、增量迁移和幂等 seed 数据
测试	JUnit 5、MockMvc、Vitest、Flutter Test、Playwright	覆盖单元、集成/API、前端逻辑和端到端业务场景
交付	Docker、GitHub Actions、Kubernetes YAML	支持镜像构建、CI/CD、健康检查、K8s 部署与 Compose 回退

3. 实践任务完成情况对照

依据任务要求，项目工作可分为原系统整理、文档与测试补齐、容器化与 CI/CD、微服务拆分、云原生实验、性能对比和综合交付管理七类。当前仓库可验证内容与文档记录如下。

要求类别	完成情况	主要证据
原系统启动与整理	已形成可运行的三端—后端基线，后端保留在 <code>services/backend</code> 作为主要可运行业务版本	<code>docs/ReadMe.md</code> 、 <code>services/backend</code> 、三端应用目录
业务场景清单	已整理 UC01-UC13 共 13 个核心业务场景，并形成需求、设计、代码与测试追溯	<code>docs/stage-new-1/业务场景用例追溯表.md</code>
需求与设计补齐	已补充用例说明、系统级/组件级/对象级建模、类图、状态图和追溯表	<code>docs/stage-new-1/UML建模与测试追溯检查版.md</code> 、 <code>docs/stage8/对象级顺序图.md</code>
单元与集成测试	已覆盖后端、Flutter 用户端、商家端和管理端的自动化测试，并纳入 CI	<code>.github/workflows/ci.yml</code> 、 <code>docs/爱团测试报告.md</code>
端到端测试	已建立 Playwright 端到端测试工程，当前可见 spec 覆盖 UC01-UC13	<code>tests/e2e/specs/*.spec.ts</code> 、 <code>.github/workflows/ci.yml</code>
容器化与 CI/CD	已配置 CI、部署 workflow、Docker 镜像构建、测试报告 artifact、K8s 和 Compose 双目标部署路径	<code>.github/workflows/ci.yml</code> 、 <code>.github/workflows/deploy.yml</code> 、 <code>k8s/README.md</code>
Kubernetes 部署	当前仓库包含单体后端版本的 K8s manifests 和部署说明，默认目标可配置为 K8s	<code>k8s/00-namespace.yaml</code> 至 <code>k8s/05-ingress.yaml</code>
微服务拆分	已完成 Maven 多模块工程骨架、服务边界、接口清单、数据表归属和拆分标准；当前分支可见四个业务微服务为启动骨架，完整业务迁移需以后续集成基线为准	<code>services/pom.xml</code> 、 <code>services/*-service</code> 、 <code>docs/stage-new-3/*.md</code>
云原生实验与性能对比	已形成 HPA、依赖故障与性能对比的实验设计和归档报告；最终引用时应以实际运行日志和原始结果包为准	<code>docs/stage-new-4/HPA与依赖故障实验说明.md</code> 、 <code>docs/stage-new-4/单体与微服务同条件三轮性能对比报告.md</code>

要求类别	完成情况	主要证据
项目管理与交付	已沉淀站会、看板、任务分工、测试报告、部署说明和答辩材料；最终归档仍需核对每日看板截图、站会提交时间和全员确认材料	docs/stage-new-1、docs/stage-final、docs/ReadMe.md

4. 需求分析与业务场景建模

4.1 用例清单整理

项目将验收范围收敛为 13 个核心业务场景，编号为 UC01-UC13。每个用例均要求包含参与者、触发条件、前置条件、主成功流程、备选或异常流程、可验证结果，并能追溯到需求、设计模型、代码模块和测试用例。

用例编号	业务场景	主要覆盖内容
UC01	用户注册、登录并进入受保护业务	认证、登录态、角色权限、受保护资源访问
UC02	用户搜索筛选商家/商品并查看详情	首页发现、搜索筛选、商家与商品详情
UC03	用户维护个人资料、地址和收藏	用户资产、地址管理、收藏状态
UC04	用户完成外卖点单、优惠结算和模拟支付	购物车、结算、优惠券、订单、支付
UC05	商家处理外卖订单并推进配送履约	接单、备餐、配送、订单状态、站内消息
UC06	用户购买非外卖服务并获得券码凭证	非配送型订单、券码生成、凭证查询
UC07	用户预约到店/团购服务并由商家核销	预约、确认、券码核销、订单完成
UC08	用户申请取消与退款	退款状态、资源释放、优惠券和券码处理
UC09	用户评价、商家回复、平台治理评价	评价发布、回复、审核、举报治理
UC10	用户提交投诉并由商家或平台处理	投诉工单、受理、处理、关闭与追踪
UC11	用户联系客服并获得 AI、商家或平台回复	客服会话、AI 降级、人工接管
UC12	用户查看会员成长值、领取并使用优惠券	会员资产、优惠券领取、抵扣与成长值
UC13	商家维护门店资料、履约规则和商品目录	商家经营、目录维护、上下架、履约规则

4.2 代表用例的模型追溯

项目没有只停留在页面或接口层描述，而是对代表用例建立从需求到测试的追溯链。以 UC04、UC05、UC12 为例：

- UC04 外卖下单支付：**通过系统顺序图描述用户端、交易服务、商家商品、优惠券、数据库和消息模块之间的调用顺序，并据此推导结算金额、订单创建、模拟支付、订单详情和权限边界断言。
- UC05 商家履约配送：**通过状态图描述订单从待支付、待接单、备餐、配送、送达到完成或退款的迁移路径，并据此设计状态变更、配送时间线和消息通知测试。
- UC12 会员优惠券：**通过顺序图描述优惠券模板创建、领取、可用券查询、下单抵扣、核销和成长值变化，并据此验证优惠金额、券状态和成长值增长规则。

这种追溯方式使 UML 图不再是展示材料，而是转化为接口顺序、状态迁移、对象职责和自动化测试断言的依据。

5. 架构设计总结

5.1 当前可运行架构

当前主要可运行业务基线采用三端一后端结构：



该架构的特点是：

- 前端按用户端、商家端、管理端拆分，减少不同角色页面和权限逻辑相互干扰；
- 后端采用 Spring Boot 统一提供认证、发现、交易、履约、会员、优惠券、评价、客服、投诉和平台治理接口；
- 数据库通过 Flyway 管理建表和 seed 数据，支持空库初始化；
- 配置和密钥通过 `.config`、`deploy/.env`、GitHub Secrets 与 Kubernetes Secret 管理，真实敏感信息不进入仓库；
- CI/CD 在部署前执行后端、Web、Flutter 和 E2E 测试，失败时停止后续发布。

5.2 微服务目标架构

为满足业务扩展和服务边界治理要求，项目设计了 Gateway + 四个业务微服务的目标架构：

服务	工程名	目标数据库	职责边界
API 网关	<code>api-gateway</code>	无业务库	统一入口、路由、请求追踪、内部接口隔离
账号与用户资产服务	<code>identity-asset-service</code>	<code>aituan_identity</code>	认证、账户、资料、地址、收藏、会员、优惠券、消息
商家与商品服务	<code>merchant-catalog-service</code>	<code>aituan_merchant</code>	商家、门店、商品、SKU、库存、搜索、履约规则
交易与履约服务	<code>trade-fulfillment-service</code>	<code>aituan_trade</code>	购物车、结算、订单、支付、退款、券码、预约、配送
互动与平台服务	<code>engagement-platform-service</code>	<code>aituan_platform</code>	评价、投诉、客服、AI、公告、配置、审计、平台看板、文件

当前代码基线中，`services/pom.xml` 已将 `common-contract`、`api-gateway`、四个业务微服务和原 `backend` 纳入 Maven 多模块工程；四个业务微服务目录已具备启动骨架。由于当前代码基线中四个微服务的业务代码尚未完整落地，业务迁移完成情况应以最终集成版本、可运行服务代码和流水线记录为准，技术总结中不将“设计文档中的完成态”替代为代码事实。

5.3 服务拆分原则

微服务拆分遵循以下原则：

- 1. **按业务能力拆分**：不按前端角色或技术层拆分，避免出现万能后台服务或数据库公共服务。
- 2. **数据所有权唯一**：每张业务表只归属一个服务，服务只能连接自己的逻辑库。
- 3. **禁止跨库耦合**：不允许跨库 JOIN、跨服务直接访问表、跨服务共享 Repository 或跨 schema 外键。
- 4. **通过接口协作**：跨服务读取通过内部接口，跨服务写入通过幂等命令和补偿机制完成。
- 5. **保存业务快照**：订单、评价、投诉、客服等跨服务历史展示保存必要快照，避免依赖远程实时联表。
- 6. **统一网关入口**：外部请求仍使用 `/api/**`，由 Gateway 按资源归属路由；`/internal/**` 不对外暴露。

5.4 跨服务调用与失败处理

以交易链路为例，交易与履约服务需要调用账号与用户资产服务获取用户、地址、优惠券和消息能力，调用商家与商品服务获取门店、商品、履约规则和库存能力。其失败处理策略包括：

- 地址、商品报价或履约规则不可用时不创建订单；
- 库存预扣失败时不继续使用优惠券；
- 优惠券使用失败时按幂等键恢复库存；
- 消息发送失败不回滚订单主状态，而是记录失败并允许重试；
- 退款补偿失败时保留补偿待处理状态，避免重复退款。

这些设计体现了微服务环境下对一致性、幂等性和补偿机制的关注。

6. 文档与追溯体系

项目文档按需求、概要设计、详细设计、阶段交付、测试部署和最终材料分层管理，并在 `docs/ReadMe.md` 中维护索引。文档体系主要包括：

- 1. **需求类文档**：描述业务目标、功能需求、非功能需求和验收范围。
- 2. **设计类文档**：包含架构设计、数据库设计、API 分组、页面路由、概要设计和详细设计。
- 3. **模型类文档**：对核心业务场景提供用例图、类图、系统级顺序图、组件级图、对象级图和状态/活动图。
- 4. **测试类文档**：记录测试计划、测试用例、测试过程、测试结果、缺陷修复和回归结论。
- 5. **部署类文档**：说明 Docker、Kubernetes、CI/CD、HTTPS、服务器配置和回滚方式。
- 6. **过程类文档**：沉淀分工、站会、看板、阶段计划、交付说明和最终答辩材料。

在追溯方式上，项目采用如下链路：

1	需求编号 / 用户故事
2	-> 业务用例
3	-> 系统级模型
4	-> 概要/组件级模型
5	-> 详细/对象级模型
6	-> 代码模块
7	-> 测试编号
8	-> 测试结果

该链路使需求变更、代码实现和测试验证之间具备可检查关系，也便于答辩或验收时从任一用例反向定位到设计依据和测试证据。

7. 测试体系总结

7.1 测试分层

项目测试体系覆盖后端、前端、端到端和部署链路，具体如下：

测试层级	工具与入口	覆盖重点
后端单元 / 集成测试	Maven、JUnit 5、Spring Boot Test、MockMvc	认证、权限、交易、优惠券、投诉、客服、评价、文件、迁移等模块
数据库迁移测试	Flyway、H2 MySQL Mode、MySQL service container	空库初始化、seed 幂等、MySQL 8 兼容性
Flutter 测试	<code>flutter analyze</code> 、 <code>flutter test</code>	模型解析、校验器、Repository、组件和页面基础回归
Web 测试	Vitest、Vite build	API 封装、Token、请求头、错误处理、构建产物
端到端测试	Playwright	UC01-UC13 业务流程，覆盖用户端、商家端、管理端和后端 API
静态回归	Bash 脚本	关键入口、文案和文件结构避免被误删
部署验证	GitHub Actions、curl、kubectl	镜像构建、应用启动、健康检查、报告归档

7.2 自动化测试现状

当前 `.github/workflows/ci.yml` 中包含以下主要 job：

- `static-regression`：执行静态回归脚本；
- `backend`：使用 Java 17 和 Maven 执行后端测试与覆盖率，并在 MySQL 8 service container 中验证 Flyway 迁移；
- `web`：分别对商家端和管理端安装依赖、执行 Vitest 覆盖率测试并构建生产产物；
- `flutter`：执行 Flutter 依赖获取、静态分析、测试与 Web 构建；
- `e2e`：启动后端与三端静态服务，执行 Playwright UC01-UC13 业务场景；
- `pipeline_raw_report`：无论成功或失败均生成流水线原始报告并上传 artifact。

该设计满足“测试失败时流水线停止”的基本要求：任一关键测试 job 失败，后续依赖 job 或部署动作不会被视为通过。

7.3 端到端测试设计

端到端测试以业务用例而不是单个按钮或接口为粒度。测试工程覆盖 UC01-UC13，对应登录、搜索、资料地址、外卖下单、履约配送、非外卖出券、预约核销、退款、评价、投诉、客服、会员优惠券、商家目录维护等完整流程。

端到端测试的价值在于：

- 验证前端、后端、数据库和测试数据能组合运行；
- 验证主成功流程和关键异常流程具备断言；
- 验证角色权限、订单状态、券码状态、消息通知和资源释放等跨模块结果；
- 为 CI/CD 提供业务级质量门禁，而不仅是编译通过。

7.4 测试报告与缺陷闭环

项目测试报告记录了测试计划、测试设计、测试用例、测试过程、测试日志、结果汇总和缺陷修复情况。主要缺陷包括 API base URL 写死、锁文件不一致、上传策略配置、金额边界误差、静态回归脚本路径过期、HTTPS 配置、调试信息暴露等，均通过修复和回归测试完成闭环。

后续测试改进方向包括：

- 提高页面级组件测试和真机端到端测试覆盖；
- 增加更稳定的覆盖率阈值和覆盖率趋势记录；
- 将生产部署日志、Kubernetes 诊断、E2E 报告和性能原始结果统一归档；
- 对搜索、下单、库存扣减、优惠券使用等高风险链路增加更细粒度异常测试。

8. 容器化、CI/CD 与 Kubernetes

8.1 容器化设计

项目保留 Docker Compose 作为回退部署方式，同时提供 Kubernetes 部署文件。当前 K8s 清单包括 namespace、ConfigMap、MySQL、后端、Web 入口、Ingress 和 Secret 示例。

Kubernetes 部署结构为：

文件	资源	作用
k8s/00-namespace.yaml	Namespace	创建 <code>aituan</code> 命名空间
k8s/01-configmap.yaml	ConfigMap	保存非敏感配置
k8s/02-mysql.yaml	StatefulSet / Service / PVC	部署 MySQL 8 并持久化数据
k8s/03-backend.yaml	Deployment / Service / PVC	部署 Spring Boot 后端和上传目录
k8s/04-web.yaml	Deployment / Service / PVC	部署 Nginx/Web 入口和下载目录
k8s/05-ingress.yaml	Ingress	暴露 Web 入口和健康检查路径
k8s/secret.example.yaml	Secret 示例	说明 Secret 字段，真实密钥不入库

8.2 CI 流水线

CI 流水线主要承担代码质量门禁，包含静态回归、后端测试、Web 测试、Flutter 测试、E2E 测试和原始报告归档。其特点是：

- 多技术栈分 job 执行，便于定位失败来源；
- 后端测试同时覆盖 H2 MySQL Mode 与 MySQL 8 迁移 smoke；
- Web 和 Flutter 均在测试通过后再构建；
- E2E 使用独立数据库容器和构建产物，避免依赖人工环境；
- 报告 artifact 可下载，支持失败追踪和复核。

8.3 CD 流水线

部署流水线支持 `k8s`、`compose` 和 `none` 三种目标，默认可配置为 K8s。主要步骤包括：

1. 校验部署变量，确保 `SERVER_ORIGIN` 为站点 origin；
2. 执行后端、Web、Flutter 和 E2E 测试；
3. 构建用户端 Web、商家端、管理端和后端产物；
4. 构建并推送后端与 Web 镜像，镜像 tag 使用提交 SHA；
5. K8s 模式下写入 kubeconfig、创建或复用 Secret、应用 YAML、设置镜像、等待 rollout；
6. Compose 模式下通过 SSH 上传 compose 文件，更新服务器环境变量并启动容器；
7. 执行集群内和公网健康检查；
8. 失败时输出 Kubernetes 或 Compose 诊断日志；
9. 上传部署流水线原始报告。

该流程体现了“先测试、后构建、再部署、最后健康检查”的持续交付顺序。

8.4 配置与密钥管理

项目明确禁止将真实密码、Token、数据库口令、云平台密钥、TLS 私钥提交到仓库。实际部署中：

- `.config` 和 `deploy/.env` 作为真实环境配置文件，不入库；
- `.config.example`、`deploy/.env.example` 和 `k8s/secret.example.yaml` 只保存字段模板；
- GitHub Actions 通过 Repository Variables 和 Production Secrets 传入服务器地址、K8s namespace、kubeconfig、数据库密码和应用配置；
- K8s 中使用 Secret 管理数据库、应用配置、镜像拉取凭据和 TLS 信息。

9. 微服务拆分与数据归属总结

9.1 拆分动机

原单体架构便于初期快速开发和联调，但随着业务模块增加，认证、商品、交易、会员、客服、评价、投诉、平台治理等逻辑逐渐耦合。微服务拆分的目标是：

1. 让业务边界更清晰，便于团队并行开发；
2. 让服务可以独立构建、测试、部署和扩缩容；

3. 限制跨领域直接访问数据库，降低隐式耦合；
4. 为故障隔离、容量管理和持续交付提供基础。

9.2 当前工程状态

当前代码基线中已完成微服务父工程和服务骨架，`services/` 目录包含：

```
1 common-contract
2 api-gateway
3 identity-asset-service
4 merchant-catalog-service
5 trade-fulfillment-service
6 engagement-platform-service
7 backend
```

其中：

- `common-contract` 用于统一响应、错误码、JWT 当前用户、请求追踪和公共枚举；
- `api-gateway` 用于网关启动骨架、健康检查和后续路由扩展；
- 四个业务微服务已建立工程目录和启动类；
- `backend` 保留原单体后端，仍是当前分支主要可运行业务实现和测试对象。

因此，本报告将微服务部分定位为“拆分设计与工程基础已具备，完整业务迁移需要以最终集成代码和运行证据确认”，避免将尚未在当前代码基线中完整落地的业务迁移写成已完成事实。

9.3 数据表归属原则

微服务数据拆分遵循四个逻辑库：

```
1 aituan_identity
2 aituan_merchant
3 aituan_trade
4 aituan_platform
```

各服务只连接自己的库，其他服务通过内部接口获取摘要、快照或执行幂等命令。典型归属如下：

- 账号、用户资料、地址、收藏、会员、优惠券、站内消息归账号与用户资产服务；
- 商家、门店、商品、SKU、库存、搜索、履约规则归商家与商品服务；
- 购物车、订单、支付记录、券码、预约、退款、配送状态归交易与履约服务；
- 评价、投诉、客服、AI 会话、公告、配置、审计、文件资产归互动与平台服务。

该归属方案避免同一业务表被多个服务同时维护，也为后续数据库权限控制和跨库访问测试提供依据。

10. 云原生实验与性能对比总结

10.1 自动扩缩容实验设计

HPA 实验设计以 Gateway 和业务服务为对象，配置 CPU request、CPU 目标和副本范围，在压力升高时触发扩容，在压力下降后自动缩容。实验报告中要求记录：

- 请求总数、成功数、错误数和错误率；
- 吞吐量、平均响应时间、P95/P99 响应时间；
- 每个服务的副本变化；
- Pod CPU、内存、事件和重启次数；
- 压测脚本、原始 JSONL 和资源时间线。

该设计避免只展示 HPA YAML，而是要求用真实业务压力证明扩缩容可触发、可观测、可恢复。

10.2 依赖故障处理设计

依赖故障实验选择商品服务不可用时的购物车场景作为示例。设计思路是：

- 正常情况下交易服务调用商品服务校验商品、价格和库存；
- 购物车读取保存商品展示快照，依赖不可用时可返回最近快照并标记降级状态；
- 新增商品、修改数量、下单等需要实时价格和库存的写操作快速失败；
- 移除或清空购物车只操作交易服务自己的数据，可继续执行；
- 商品服务恢复后自动恢复实时校验，不要求重启交易服务。

该方案体现故障隔离的基本目标：依赖服务故障不应导致全系统崩溃，也不能在状态未知时继续接受高风险写入。

10.3 性能对比方法

性能对比要求选择 2-3 个主要接口，在单体和微服务版本中使用同一机器、同一批数据、同一压力脚本，各运行至少 3 次。归档报告采用首页、推荐列表和门店搜索三个只读接口，记录吞吐、平均响应时间、P95、P99、错误率、CPU 和内存。

根据归档报告，测试强调以下口径：

1. 响应数据需做等价校验，避免因数据量不同造成虚假优势；
2. HPA 固定为 1 副本，避免扩容能力干扰架构对比；
3. 单体与微服务均记录 CPU 和内存，既看性能收益，也看资源代价；
4. 结论不能泛化为“微服务天然更快”，只能限定在相同测试条件和选定接口下解释。

11. 项目管理与协作总结

项目过程管理主要围绕需求拆解、分工协作、每日同步、看板跟踪、流水线反馈和阶段交付进行。

11.1 分工方式

系统按业务模块和工程能力进行分工，典型职责包括：

- 用户端与基础体验；
- 交易、履约、会员和优惠券；
- 商家端和商品服务；
- 管理端、部署和 CI/CD；
- 评价、客服、投诉、AI 和平台治理。

在微服务阶段，分工进一步收敛为一人或一组负责一个业务服务，并共同维护 Gateway、数据库拆分、Docker/K8s 和 CI/CD 统一标准。

11.2 看板和过程证据

项目文档中保留了任务分工、阶段计划、站会记录、交付说明和最终答辩材料。过程管理的价值在于：

1. 将业务场景拆为可跟踪任务；
2. 为每项任务定义负责人、完成标准和证据；
3. 将提交、测试、文档和流水线结果关联起来；
4. 支持中期检查、最终验收和成员贡献说明。

最终归档时仍建议人工核对每日站会、看板截图、云盘提交时间、全员确认记录和答辩材料，保证过程证据与仓库代码一致。

11.3 质量控制流程

项目实践中形成了较明确的质量控制流程：

1

需求确认 -> 设计建模 -> 编码实现 -> 单元/API测试 -> E2E回归 -> CI门禁 -> 镜像构建 -> 部署验证 -> 文档归档

该流程避免直接以功能截图作为验收依据，而是要求代码、测试、部署日志和文档共同支撑结论。

12. AI 工具使用说明

项目过程中使用生成式 AI 工具辅助完成以下工作：

1. 梳理需求、用例、追溯表和技术文档草稿；
2. 辅助分析接口边界、微服务拆分方案和测试覆盖缺口；
3. 生成或改写部分测试用例、脚本说明和部署检查清单；
4. 协助排查构建、测试和部署过程中的错误信息。

AI 工具仅作为辅助工程工具使用，不作为最终正确性的来源。所有关键结论均需由团队成员根据仓库代码、运行结果、测试报告、流水线日志和部署环境进行人工复核；涉及密钥、生产配置、删除、提交、推送和部署等高风险操作时，仍需人工确认。项目文档和代码中不保存真实密码、Token、数据库口令或云平台密钥。

13. 主要成果

综合当前仓库和文档，项目主要成果包括：

- 1. 建立了本地生活服务平台的完整需求、设计、测试和交付文档体系；
- 2. 梳理了 UC01-UC13 业务场景，并形成需求、模型、代码、测试的追溯关系；
- 3. 实现了用户端、商家端、管理端和后端服务的主业务闭环；
- 4. 后端、Flutter、Vue Web 和 Playwright 端到端测试均纳入自动化流水线；
- 5. 形成了 Docker、Kubernetes、GitHub Actions、Secret 管理和回滚说明；
- 6. 完成了微服务拆分设计、Maven 多模块工程骨架和服务边界规范；
- 7. 形成了 HPA、故障隔离和性能对比的实验方案与结果归档材料；
- 8. 建立了问题修复、回归验证、测试报告和流水线原始报告留存机制。

14. 不足与改进方向

项目仍存在以下需要继续完善的方面：

类别	不足	改进方向
微服务实现	当前代码基线中四个业务微服务主要为启动骨架，业务迁移完成情况需以最终集成基线确认	将单体业务按服务边界逐步迁移，补齐各服务 Controller、Service、Repository、Flyway 和测试
端到端测试	已有 UC01-UC13 spec，但仍需持续维护测试数据隔离和报告归档	固化测试数据初始化、失败截图、trace、HTML 报告和 CI artifact
页面测试	Web 页面级组件测试和 Flutter 真机自动化仍可增强	增加 Vitest 组件测试、Playwright 页面断言和 Flutter integration_test
云原生实验	HPA、故障、性能材料需和实际运行日志保持一致	最终归档原始 JSONL、kubectl 输出、资源采样、测试配置和失败重跑记录
性能分析	当前性能对比主要覆盖只读接口	后续增加下单、支付、退款等写链路压力测试，并用 profiler 定位热点
运维治理	证书续期、Secret 轮换、日志告警仍可增强	增加续期脚本、Secret 更新流程、日志告警和部署失败演练说明

15. 结论

爱团本地生活服务综合平台围绕本地生活服务主题，完成了从需求分析、业务建模、系统设计、编码实现、自动化测试、容器化部署到持续交付的工程闭环。项目的核心价值不只在实现若干业务功能，而在于通过用例追溯、分层测试、CI/CD 门禁、Kubernetes 部署方案和微服务边界设计，体现软件工程过程的规范性、可复现性和可验证性。

当前可运行基线以三端和单体后端为主，微服务方向已经具备清晰的服务划分、数据归属、接口契约和工程骨架。后续若继续推进，应以现有追溯表和测试体系为约束，逐步完成业务代码迁移、跨服务契约验证、真实部署和实验结果归档，确保系统在架构演进过程中保持可运行、可测试、可回滚和可说明。